Memoria conceptual

Reflexión sobre arquitectura, pruebas y control de versiones

JAIME MONTALVO

M1. Familias de fallo de arquitectura

- Dinámicas: El flujo de mensajes entre dos nodos no es el adecuado, así como puede serlo también su contenido, por ejemplo, que un nodo, como un sensor publique mucho más lento de lo esperado. Esto puede detectarse con (ros2 topic hz) (ros2 topic echo /nombre topic).

- Estructural: Errores humanos, causados durante la programación, que dos simuladores publiquen en el mismo sitio (ros2 topic info -v). Que se apliquen QoS incompatibles, que un nodo no sea encontrado (ros node list). etc.

- De tiempo: Los mensajes llegan sistemáticamente tarde o cuando no toca, esto puede estar causado por un medio de comunicación no apropiado, un elevado tiempo de procesamiento o transformaciones TF incorrectas(ros2 run tf2_ros) (ros2 topic delay).

- Físicos: Que el fallo ocurra por causas ajenas a la arquitectura diseñada (si es que eso existe). Por ejemplo, que un sensor entregue valores incorrectos (ros2 topic echo). Aquí se puede requerir de intervención física (ros2 topic info -v /nombre_topic).

M2. Importancia de las pruebas unitarias

Una herramienta de análisis necesita pruebas unitarias para comprobar que cada función detecta correctamente los casos previstos y no da resultados erróneos. Sin ellas, se pueden obviar errores de ejecución o no cumplir exigencias que degraden el servicio o causen un colapso total del sistema si son aplicados. Por ejemplo, si un analizador de bags no detecta mensajes fuera de rango, podría considerar válida una grabación incorrecta. Decir que «un fallo no detectado es una especificación pendiente» significa quesi descubrimos un caso que la herramienta aún no contempla, debemos definir cómo debería comportarse y añadir una prueba que lo compruebe, de esta forma vamos aumentando la seguridad de nuestros cambios a medida que iteramos.

M3. Conflictos de fusión en Git

Un conflicto ocurre cuando Git no combina automáticamente dos cambios incompatibles realizados sobre la misma parte de un archivo. Se muestra mediante marcas nombre-rama, que separan las dos versiones. Para resolverlo hay que leer ambas, decidir qué contenido debe conservarse, editar el archivo eliminando las marcas y realizar el commit de la fusión. No se debe «aceptar el mío» sin leer porque podríamos descartar cambios válidos de otra rama o eliminar una corrección necesaria. Además, si trabajamos en un equipo, puede aportar valor comparando ambas propuestas y decidirse por una o fusionar ambas.

M4. Qué garantiza una CI en verde

Una CI en verde garantiza que las comprobaciones automatizadas configuradas se han ejecutado previamente en el entorno git sobre los cambios evaluados de forma correcta.Pero no demuestra que el programa esté libre de errores ni que se hayan probado todos los escenarios reales. Por ejemplo, en esta misma práctica, en el contrato de regresión con bags, una CI en verde indica que las grabaciones analizadas cumplen las condiciones definidas en las pruebas, como los topics esperados o sus rangos. Pero no garantiza que se hayan contemplado todos los fallos posibles ni que el sistema funcione correctamente en cualquier entorno.